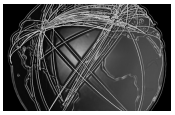


Sistemas distribuidos - Nombrado en SD

Contenido

Objetivos 1
Objetivo general 2
Objetivos específicos 2
Bibliografía 2
Preguntas 2
Nombre, identificador y direccion 2
Resolución de nombres 2
Espacios de nombres 3
DNS 3
Nombrado en arquitecturas modernas 3
4. Ejercicios 4
Caso práctico: Plataforma distribuida de comercio electrónico 4
Ejercicio 1 - Crear una red Docker 4
Ejercicio 2 - Crear el servicio de inventario 4
Ejercicio 3 - Crear un cliente 5
Ejercicio 4 - Cambio de dirección 5
Ejercicio 5 - DNS tradicional 5
Ejercicio 6 - InventoryService en C# 5
Ejercicio 7 - OrdersService 6
Ejercicio 8 - Dockerizar los servicios 6
Ejercicio 9 - Docker Compose 7
Ejercicio 10 - Recrear la infraestructura 7
Ejercicio 11 - Multiples instancias 7
Ejercicio 12 - Service Discovery 8
Ejercicio 13 - Kubernetes 8
Ejercicio 14 - Comparacion 9
Ejercicio 15 - Conclusión 9

Objetivos



Objetivo general

Aplicar los conceptos teóricos relacionados con el **nombrado en sistemas distribuidos** a situaciones prácticas actuales, analizando como los nombres permiten identificar, localizar y acceder a recursos distribuidos sin depender directamente de su ubicación física o de sus direcciones de red.

Objetivos específicos

- Comprender la diferencia entre **nombre, identificador y dirección**.
- Comprender el concepto de **resolución de nombres**.
- Analizar la importancia de separar el nombre lógico de un recurso de su ubicación física.
- Comprender el funcionamiento de los **espacios de nombres**.
- Analizar mecanismos de nombrado plano y jerárquico.
- Comprender el funcionamiento de **DNS** como sistema distribuido de nombres.
- Experimentar con resolución de nombres mediante **Docker**.
- Relacionar nombrado con **microservicios y Service Discovery**.
- Analizar como el nombrado facilita escalabilidad, movilidad y transparencia de ubicación.

Bibliografía

1. Tanenbaum, A. S. y Van Steen, M. *Distributed Systems*. Consultar los contenidos sobre nombres, identificadores, direcciones, espacios de nombres, resolucion de nombres y servicios de directorio.
2. Coulouris, G., Dollimore, J., Kindberg, T. y Blair, G. *Distributed Systems: Concepts and Design*. Pearson.
3. Docker Documentation. Redes de contenedores, DNS interno y comunicacion entre servicios.
4. Microsoft .NET Documentation. ASP.NET Core, HttpClient, Web API y contenedores.
5. Kubernetes Documentation. Services, DNS for Services and Pods y Service Discovery.

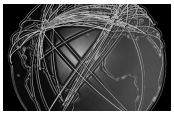
Preguntas

Nombre, identificador y direccion

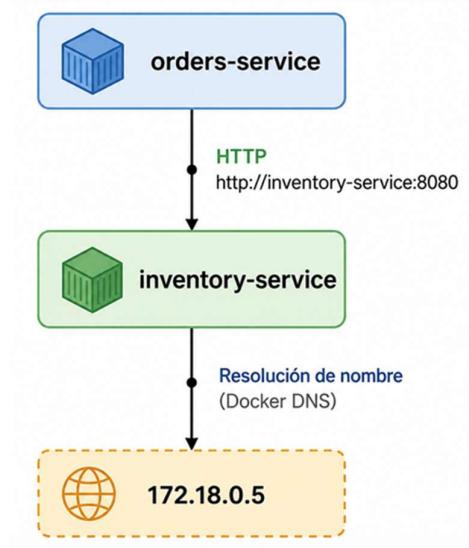
Considere el siguiente nombre y dirección: y responder las preguntas

Nombre: `inventory-service` Direccion: `172.18.0.5`	1. ¿Qué diferencia existe entre el nombre inventory-service y la dirección 172.18.0.5? 2. ¿Qué se entiende por nombre dentro de un sistema distribuido? 3. ¿Qué se entiende por identificador? 4. ¿Qué se entiende por dirección? 5. ¿Una dirección puede utilizarse como identificador? Justifique. 6. ¿Puede una misma entidad tener diferentes nombres? Proporcione un ejemplo. 7. ¿Puede un mismo nombre representar diferentes instancias de un servicio?

Resolución de nombres



Considere el siguiente esquema y responder las preguntas:



1. ¿Qué significa resolver un nombre?
2. ¿Qué componente transforma inventory-service en una direccion?
3. ¿Por qué conviene utilizar nombres en lugar de IP configuradas directamente?
4. ¿Qué ocurre si inventory-service cambia su direccion IP?
5. ¿Que relacion existe entre resolucion de nombres y transparencia de ubicacion?

Espacios de nombres

Considere: `api.inventory.empresa.com`

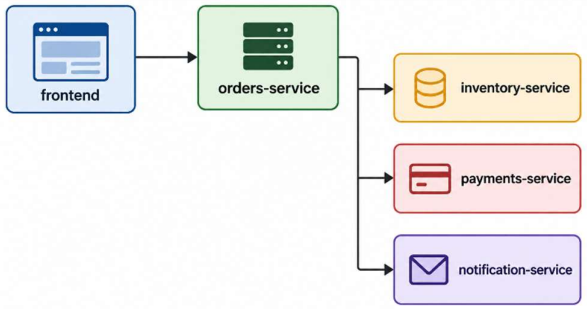
1. ¿Qué es un espacio de nombres?
2. ¿Corresponde a un esquema plano o jerárquico?
3. Identifique los diferentes niveles del nombre.
4. ¿Qué ventajas presenta un espacio de nombres jerárquico?
5. ¿Por qué DNS utiliza una estructura jerárquica?

DNS

1. ¿Que función cumple DNS?
2. ¿Por qué DNS puede considerarse un sistema distribuido?
3. ¿Que información almacena un servidor DNS?
4. ¿Qué diferencia existe entre un nombre de dominio y una dirección IP?
5. ¿Qué ventajas proporciona el uso de cache?
6. ¿Que ocurriría si las aplicaciones tuvieran que conocer directamente las IP de todos los servicios?

Nombrado en arquitecturas modernas

Considere la siguiente grafica y responda las preguntas:



1. ¿Por que resulta inconveniente configurar manualmente las IP de los microservicios?
2. ¿Que ocurre cuando un contenedor es eliminado y creado nuevamente?
3. ¿Por que un nombre puede permanecer estable aunque cambie la direccion?
4. ¿Que se entiende por Service Discovery?
5. ¿Que relacion existe entre Service Discovery y nombrado?
6. ¿Que diferencias existen entre DNS tradicional y descubrimiento de servicios?

4. Ejercicios

Caso práctico: Plataforma distribuida de comercio electrónico

Una empresa posee una plataforma compuesta por:

- frontend
- orders-service
- inventory-service
- payments-service
- notification-service
- postgres-orders
- postgres-inventory

Inicialmente orders-service utiliza: `http://172.18.0.5:8080/api/inventory`

Luego de recrear el contenedor, inventario recibe: 172.18.0.9

Se propone utilizar: `http://inventory-service:8080/api/inventory`

Ejercicio 1 - Crear una red Docker

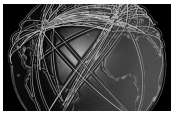
```
docker network create ecommerce-network
docker network ls
docker network inspect ecommerce-network
```

Analizar:

1. ¿Que función cumple una red Docker?
2. ¿Que servicios pueden comunicarse en ella?
3. ¿Que relación tiene con la resolución de nombres?

Ejercicio 2 - Crear el servicio de inventario

```
docker run -d --name inventory-service --network ecommerce-network nginx
docker ps
```



```
docker inspect inventory-service
```

Registrar nombre, IP, red y estado. Analizar la diferencia entre nombre y direccion.

Ejercicio 3 - Crear un cliente

```
docker run -it --name orders-service --network ecommerce-network alpine
```

Dentro del contenedor:

```
apk add --no-cache bind-tools curl
ping inventory-service
nslookup inventory-service
getent hosts inventory-service
```

Analizar quien resolvió el nombre, si fue necesario conocer la IP y que ventaja presenta.

Ejercicio 4 - Cambio de dirección

```
docker stop inventory-service
docker rm inventory-service
docker run -d --name inventory-service --network ecommerce-network nginx
docker inspect inventory-service
```

Desde orders-service:

```
nslookup inventory-service
```

Responder:

1. ¿Cambio la IP?
2. ¿Cambio el nombre?
3. ¿Fue necesario modificar el cliente?
4. ¿Que relacion tiene con transparencia de ubicacion?

Ejercicio 5 - DNS tradicional

```
nslookup www.google.com
```

Opcional:

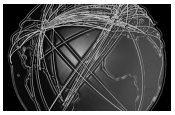
```
dig www.google.com
```

Comparar DNS de Internet con la resolucion interna de Docker.

Ejercicio 6 - InventoryService en C#

Crear una Web API:

```
dotnet new webapi -n InventoryService --use-controllers
cd InventoryService
```



Ejemplo de controlador:

```
using Microsoft.AspNetCore.Mvc;

namespace InventoryService.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class InventoryController : ControllerBase
    {
        [HttpGet("{productId}")]
        public IActionResult Get(string productId)
        {
            return Ok(new
            {
                ProductId = productId,
                Stock = 25,
                Server = Environment.MachineName
            });
        }
    }
}
```

Ejecutar:

```
dotnet run
```

Ejercicio 7 - OrdersService

Crear:

```
dotnet new webapi -n OrdersService --use-controllers
```

El cliente debera utilizar el nombre:

```
var client = new HttpClient();

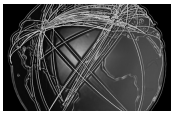
var response = await client.GetAsync(
    "http://inventory-service:8080/api/inventory/P001");

var content = await response.Content.ReadAsStringAsync();
              Console.WriteLine(content);
```

Comparar el uso de una IP directa con el nombre logico del servicio.

Ejercicio 8 - Dockerizar los servicios

Dockerfile de InventoryService:



```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
COPY . .
RUN dotnet restore
RUN dotnet publish -c Release -o /app/publish

FROM mcr.microsoft.com/dotnet/aspnet:8.0
WORKDIR /app
COPY --from=build /app/publish .
ENTRYPOINT ["dotnet", "InventoryService.dll"]
```

Crear un Dockerfile equivalente para OrdersService.

Ejercicio 9 - Docker Compose

Crear docker-compose.yml:

```
services:
  inventory-service:
    build: ./InventoryService
    networks:
      - ecommerce-network

  orders-service:
    build: ./OrdersService
    depends_on:
      - inventory-service
    networks:
      - ecommerce-network

networks:
  ecommerce-network:
    driver: bridge
```

Ejecutar:

docker compose up --build
docker compose ps

Analizar nombres, direcciones y resolucion.

Ejercicio 10 - Recrear la infraestructura

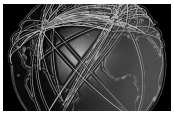
```
docker compose down
docker compose up -d
```

Consultar las direcciones antes y despues con:

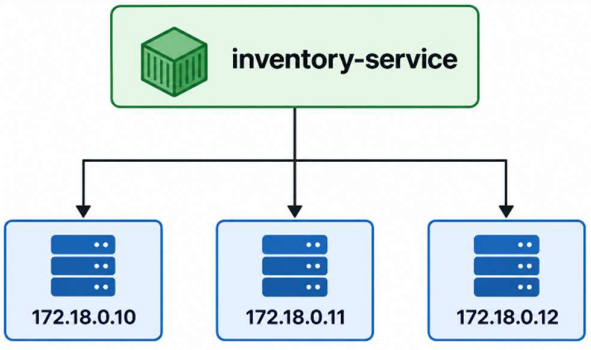
```
docker inspect <nombre-del-contenedor>
```

Determinar si el cliente necesita modificaciones cuando cambia la IP.

Ejercicio 11 - Multiples instancias



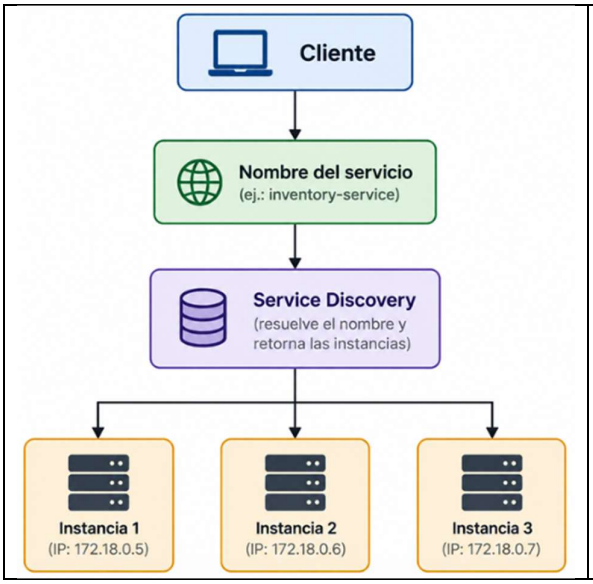
Analizar y responder las preguntas:



1. ¿Deberia OrdersService conocer las tres IP?
2. ¿Que problemas generaria?
3. ¿Que mecanismo deberia localizar las instancias?
4. ¿Como se relaciona con balanceo de carga?
5. ¿Que ocurre si una instancia falla?

Ejercicio 12 - Service Discovery

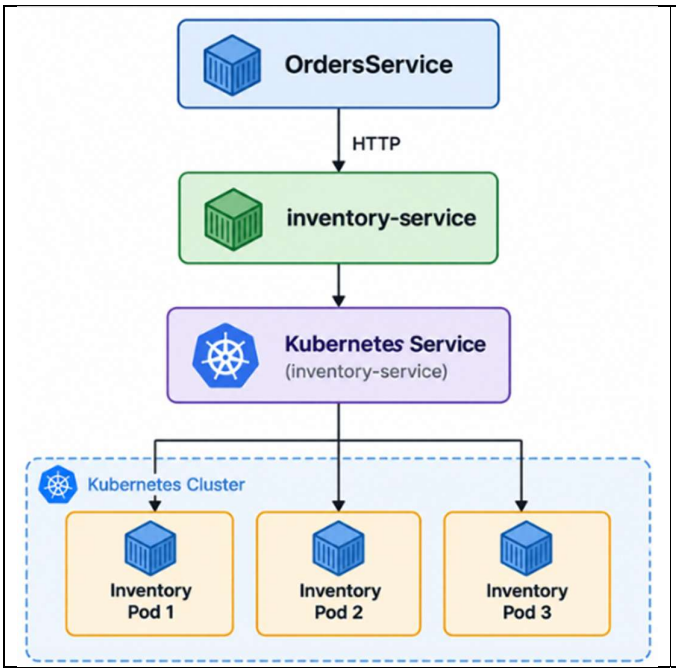
Analizar y responder las preguntas:



1. ¿Que información mantiene Service Discovery?
2. ¿Como se registran nuevas instancias?
3. ¿Como se detectan instancias no disponibles?
4. ¿Que relación existe con nombrado?
5. ¿Por que es importante en microservicios?

Ejercicio 13 - Kubernetes

Analizar y responder las preguntas:



1. ¿Que problema de nombrado resuelve Kubernetes Service?
2. ¿Por que no conviene referenciar directamente las IP de los Pods?
3. ¿Que ocurre si un Pod es reemplazado?
4. ¿El cliente necesita conocer el cambio?
5. ¿Como se relaciona con transparencia de ubicacion?

Ejercicio 14 - Comparacion

Mecanismo	Nombre utilizado	Quien resuelve	Destino	Puede cambiar la IP
DNS Internet	www.ejemplo.com	DNS	Servidor	Si
Docker	inventory-service	DNS interno Docker	Contenedor	Si
Kubernetes	inventory-service	DNS / Service	Pods	Si
IP directa	172.18.0.5	No existe resolucion	Host/Contenedor	La referencia queda obsoleta

Ejercicio 15 - Conclusión

Elaborar una conclusion tecnica relacionando:

NOMBRE→ RESOLUCION→ DIRECCION→ ENTIDAD / SERVICIO

Explicar por que los nombres son fundamentales en sistemas distribuidos cuyos componentes pueden cambiar de ubicacion, reiniciarse, escalar horizontalmente o ser reemplazados sin modificar los clientes.